CNN Practicals

Structured Notes

Complete CNN Pipeline, Training with

Cross Entropy Loss,
Dropout Regularization, and Practical Demonstrations

Contents

1 Complete CNN Pipeline Visualization **2**
1.1 Feature Map Visualization Through Layers . . . 2
1.1.1 After Convolution (Before ReLU) . . . 2
1.1.2 After ReLU Activation . . . 2
1.1.3 After Pooling . . . 2
1.1.4 After Flattening . . . 2
1.1.5 After Fully Connected Layer . . . 2

2 Convolution vs Pooling Operations **3**

3 Output Size Formula **3**
3.1 Concept Check Examples . . . 3

4 Kernel Size and Weight Sharing **3**
4.1 Parameter Count . . . 4

5 Training a CNN **4**
5.1 Cross Entropy Loss . . . 4
5.2 Softmax Activation Function . . . 5
5.3 Backpropagation in CNNs . . . 5

6 CNN Training Behavior **5**
6.1 Why CNNs Can Learn Nonlinear Decision Boundaries . . . 5
6.2 Challenges in Training Deeper CNNs . . . 6
6.3 Momentum and Adam Optimizer . . . 6

7 Double Descent Phenomenon **6**

8 Overfitting in CNNs **6**
8.1 Causes of Overfitting . . . 6
8.2 Overfitting Signature . . . 7
8.3 Guidelines for Small Datasets . . . 7

9 Transfer Learning (Overview) **7**

10 Dropout Regularization **7**
10.1 How Dropout Works . . . 7
10.2 Mathematical Formulation . . . 7
10.3 Where to Apply Dropout in CNNs . . . 8
10.4 Training vs Evaluation Behavior . . . 8
10.5 Why Dropout Works . . . 8

11 Practical Demonstration: CNN With vs Without Dropout **8**
11.1 Model Setup . . . 9
11.2 Key Observations . . . 9
11.2.1 Training Mode (`model.train()`) . . . 9
11.2.2 Evaluation Mode (`model.eval()`) . . . 9
11.3 Loss Curve Comparison . . . 9

12 Summary **9**

1 Complete CNN Pipeline Visualization

CNN Pipeline Stages

The complete CNN pipeline processes an image through six distinct stages:

1. **Data Preparation** — Prepare the raw image data
2. **Tensor Conversion** — Convert data into tensors
3. **Convolution Layers** — Pass through convolution + ReLU + pooling blocks
4. **Flattening** — Convert the 2D feature maps into a 1D vector
5. **Dense (Fully Connected) Layers** — Each neuron connected to every neuron in the next layer
6. **Output** — Logits converted to probabilities via softmax activation

The pipeline for a 2-layer CNN follows this flow:

Input → Conv1 → ReLU1 → Pool1 → Conv2 → ReLU2 → Pool2 → Flatten → FC Layer → Output (Logits)

In general, CNNs can have N convolution layers stacked sequentially. After all convolution blocks, the output must be flattened before passing to dense layers.

1.1 Feature Map Visualization Through Layers

1.1.1 After Convolution (Before ReLU)

When an RGB image (3 channels) passes through the first convolution layer with 16 filters, it produces 16 feature maps (channels). Each filter extracts different features from the input. The original 3-channel RGB representation is transformed into 16 channels containing different feature information—no longer just RGB.

1.1.2 After ReLU Activation

After applying ReLU, some activations pass through while others are zeroed out. This creates a binary-like effect: values above zero are retained, values below zero become zero.

1.1.3 After Pooling

After max pooling, the spatial dimensions are halved but the number of channels remains the same (still 16). The resolution is reduced—the same area that was previously captured by 4 pixels is now represented by a single pixel. The information content per pixel increases, but the overall resolution decreases.

1.1.4 After Flattening

After flattening, the 2D feature maps are converted into a 1D vector. For example, with shape $1 \times 32 \times 8 \times 8$, the flattened output contains $32 \times 8 \times 8 = 2048$ values. Each cell in this vector holds a numerical value representing extracted features.

1.1.5 After Fully Connected Layer

The 2048-element vector is transformed to a smaller vector (e.g., 64 neurons) through the operation $W^T x + b$. After applying ReLU, all negative values become zero, and only the remaining non-zero vectors carry forward to produce the final logits.

2 Convolution vs Pooling Operations

Aspect	Convolution	Pooling
Parameters	Has learnable weights	No parameters, no weights
Feature Creation	Creates new features through learned filters	Does not create new features
Operation	Weighted sum with learned kernel	Selects maximum value (max pooling)
Size Reduction	Can reduce size by increasing stride	Reduces size by design
Purpose	Feature extraction and transformation	Dimensionality reduction

Stride and Output Size

Output size reduction is not exclusive to pooling. Increasing the stride value in a convolution layer also reduces the output spatial dimensions. The key difference is that convolution uses learnable parameters while pooling does not.

3 Output Size Formula

The output spatial dimension after a convolution or pooling operation:

$$\text{Output Size} = \frac{H - K + 2P}{S} + 1$$

where:

- H = input height (or width)
- K = kernel size
- P = padding
- S = stride

3.1 Concept Check Examples

Example 1: Input size $30 \times 30 \times 3$, Conv2D with `out_channels=16`, kernel size $K = 3$, stride $S = 1$, padding $P = 0$:

$$\text{Output} = \frac{30 - 3 + 0}{1} + 1 = 28 \quad \Rightarrow \quad \text{Output shape: } 28 \times 28 \times 16$$

Example 2: After MaxPool2D with size 2×2, stride 2:

$$\text{Output} = \frac{28 - 2}{2} + 1 = 14 \quad \Rightarrow \quad \text{Output shape: } 14 \times 14 \times 16$$

Height and width are halved; channels remain the same.

Example 3: If stride is changed to 2 in the first convolution (all else same):

$$\text{Output} = \frac{30 - 3 + 0}{2} + 1 = 14.5 \to 14 \quad \Rightarrow \quad \text{Output size is halved}$$

4 Kernel Size and Weight Sharing

Filter/Kernel Dimensions

The size of a single convolutional kernel is not just $K \times K$—it also depends on the number of input channels:

$$\text{Kernel size} = C_{\text{in}} \times K \times K$$

For an RGB input with a 3×3 kernel: $3 \times 3 \times 3 = 27$ weights per filter. Each filter operates across all input channels and produces a single feature map. With 16 filters, the layer produces 16 output channels.

Weight Sharing

Weight sharing is a key property of CNNs: the same filter (with the same weights) is applied across all spatial locations of the input. This means:

- Fewer parameters compared to fully connected networks
- The same kernel "rolls over" all edges and regions of the image
- Convolution gradients accumulate over spatial locations during backpropagation

Different filters have different weights. If 32 filters are used, all 32 have independent weight values. However, a single filter maintains the same weights as it slides across the image (weight sharing).

4.1 Parameter Count

The convolutional layer parameter count does **not** depend on the input height H and width W—it depends only on:

- Number of input channels (C_{in})
- Number of output channels/filters (C_{out})
- Kernel size (K)

$$\text{Parameters} = C_{\text{out}} \times (C_{\text{in}} \times K \times K + 1)$$

The $+1$ accounts for the bias term per filter.

The layer with the **highest parameter count** in a CNN is typically the **first fully connected layer**, because flattening creates a very long vector that connects to every neuron in the FC layer.

5 Training a CNN

Three essential components are required for training:

1. **Model** — The CNN architecture (already defined)
2. **Loss Function** — Measures prediction error
3. **Optimizer** — Updates weights to minimize loss

5.1 Cross Entropy Loss

Cross Entropy Loss Types

Two types of cross entropy loss:

- **Binary Cross Entropy Loss** — For binary classification (uses sigmoid activation, two outputs)
- **Cross Entropy Loss** — For multi-class classification (uses softmax activation, N outputs)

5.2 Softmax Activation Function

Softmax converts raw logits (values in $(-\infty, +\infty)$) into probabilities for each class. For N classes:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}$$

Properties:

- Each output is a probability between 0 and 1
- The sum of all output probabilities equals 1: $\sum_{i=1}^{N} P(z_i) = 1$
- The class with the highest probability is the predicted class

Sigmoid produces only two outputs (binary), while softmax produces probabilities for all N classes simultaneously.

PyTorch Cross Entropy Loss

In PyTorch, `nn.CrossEntropyLoss` combines `LogSoftmax` and `NLLLoss` (Negative Log Likelihood) internally. Therefore, **raw logits** must be fed directly to the loss function—**not** the softmax output. Applying softmax before passing to `CrossEntropyLoss` would apply softmax twice.

This is specific to PyTorch. In TensorFlow, the behavior differs and softmax may need to be applied explicitly.

5.3 Backpropagation in CNNs

Backpropagation in CNNs follows the same principle as in ANNs:

1. Forward pass produces predictions (logits)
2. Loss is computed between predicted and actual labels
3. Gradients are computed via chain rule
4. Weights are updated using the optimizer

Even without training, convolution creates new features from the input. While creating new features, new errors are also introduced. There is always a chance of misclassification, which is measured as the loss. The loss is then reduced by updating weights through backpropagation.

6 CNN Training Behavior

Key observations during CNN training:

- Loss decreases across iterations
- Gradients exist for both convolution weights and fully connected weights
- Convolution gradients accumulate over spatial locations due to weight sharing—as the kernel moves across the image, gradients are summed at every spatial position
- Gradient density and magnitudes may differ across layers

CNN vs ANN Optimization

CNN optimization is **not fundamentally different** from ANN optimization. The difference lies in the **parameter structure** (shared weights, spatial operations), not in the learning rule itself. Both use gradient descent with backpropagation.

6.1 Why CNNs Can Learn Nonlinear Decision Boundaries

Despite convolution being a linear operation, CNNs learn nonlinear decision boundaries because of **activation functions** (ReLU). Without ReLU, stacking multiple convolution layers would be equivalent

to a single linear transformation.

6.2 Challenges in Training Deeper CNNs

Deeper CNNs can be harder to train due to:

1. **Vanishing gradients** — Gradients become very small as they propagate through many layers
2. **Overfitting** — More parameters increase the risk of memorizing training data
3. **Optimization instability** — Loss landscape becomes more complex with depth

6.3 Momentum and Adam Optimizer

To address training instability, momentum is introduced through optimizers like **Adam**. Adam incorporates momentum to smooth out gradient updates and help the optimization process converge more reliably.

7 Double Descent Phenomenon

Double Descent

Double descent is a phenomenon observed in many models where the loss initially decreases, then increases (shoots up), and then decreases again. This creates a characteristic "double dip" in the loss curve.

This phenomenon is analogous to human learning:

1. **Initial partial learning** — Learn something new partially, apply it, and get reasonable results (low loss)
2. **Overconfidence and failure** — As learning proceeds, overconfidence leads to mistakes (loss increases)
3. **Deeper understanding** — After recognizing the complexity, deeper learning occurs and performance improves again (loss decreases)

Previously, the initial loss spike was simply attributed to the model "still learning." Double descent provides a more structured explanation for this behavior. The key practical insight: what matters is the **final converged loss**, not the intermediate fluctuations.

8 Overfitting in CNNs

CNNs have fewer parameters than a fully connected ANN applied to images, but they can still overfit.

8.1 Causes of Overfitting

- The model can **memorize patterns** specific to the training set
- Fully connected layers near the end can become **memorization-heavy**
- If the dataset is **small** or labels are **noisy**, features can become overly specialized

Noisy Labels

Noisy labels refer to situations where features **overlap between classes**. When similar features appear across multiple classes (e.g., the same features belong to class 1, class 2, and class 3), it becomes very difficult for the model to predict the correct class. The model must find class-specific features; if such features do not exist, classification becomes impossible.

Example: Distinguishing between the characters "2" and "Z" — some handwriting styles make these nearly identical, creating feature overlap and noisy labels.

8.2 Overfitting Signature

- **Training loss** keeps decreasing
- **Validation loss** stops improving (or starts increasing)

8.3 Guidelines for Small Datasets

Small Dataset Strategy

When the dataset is small, training a custom small CNN with fewer layers is better than using transfer learning. Specific guidelines:

- Use smaller filters and fewer filters per layer (e.g., 8 or 16 instead of 32)
- Increase depth (more layers) but reduce width (fewer filters per layer)
- Minimum recommended: approximately 5,000 images (e.g., 5 classes with 1,000 images each)
- Increasing layers with insufficient data points will cause overfitting

9 Transfer Learning (Overview)

Transfer Learning

Transfer learning involves taking a model that has been pre-trained on a very large dataset (with billions of parameters by companies with significant computational resources) and fine-tuning it on a smaller target dataset. The pre-trained weights are already optimized, so less computation and fewer optimization steps are needed to achieve good performance on the new data.

Transfer learning is particularly useful when computational resources (GPUs) are limited. However, for very small datasets, transfer learning can lead to overfitting because the pre-trained models were designed for large-scale data.

10 Dropout Regularization

Dropout

Dropout is one of the most effective regularization techniques. It works by randomly turning off (setting to zero) a fraction of neuron activations during training. This prevents the network from over-relying on any single neuron and promotes generalization.

10.1 How Dropout Works

If the dropout probability is P:

- With probability P, a neuron's output becomes zero (turned off)
- With probability $1 - P$, the neuron remains active

Example: If $P = 0.2$ and there are 100 neurons:

- Each neuron has a 20% chance of being turned off
- Approximately 20 neurons will be turned off, 80 will remain active
- The specific neurons turned off are **randomly selected** each time

The dropout probability is a **hyperparameter**. Typical range: 0.2 to 0.5.

10.2 Mathematical Formulation

Dropout uses a Bernoulli random variable M:

$$M \sim \text{Bernoulli}(1 - P)$$

The Bernoulli probability mass function:

$$P(K = k) = p^k \cdot (1 - p)^{1-k}, \quad k \in \{0, 1\}$$

The dropout operation:

$$\tilde{H} = M \odot H$$

where H is the original activation vector and $\odot$ denotes element-wise multiplication. Each neuron is independently either kept ($k = 1$) or dropped ($k = 0$).

10.3 Where to Apply Dropout in CNNs

Location	Impact	Usage
Before/between FC layers	Strongest impact	Recommended
After flatten layer	Strong impact	Common beginner-friendly rule
After convolution blocks	Optional	Less common in small CNNs

Beginner-friendly rule: Apply dropout after the flatten layer and after the first fully connected layer.

Dropout is applied on dense/FC layers

Dropout is applied on the linear (dense/fully connected) layers, **not** on the convolution layers. It can optionally be applied after convolution blocks, but this is less common in small CNNs.

10.4 Training vs Evaluation Behavior

Dropout During Training vs Evaluation

- **During training:** Dropout is **ON** — neurons are randomly turned off to prevent overfitting
- **During evaluation/testing:** Dropout is **OFF** — all neurons are active, and the model uses all learned weights

In PyTorch:

- `model.train()` — sets dropout ON
- `model.eval()` — sets dropout OFF

Once the model is trained, dropout is no longer needed. All weights are set and the model is ready for inference.

10.5 Why Dropout Works

- Forces the network to avoid relying too much on any single neuron
- Creates randomness in the network, reducing over-dependency on specific high-output neurons
- Promotes **generalization** by ensuring multiple neurons learn useful representations
- Dropout is primarily needed when facing significant overfitting issues

11 Practical Demonstration: CNN With vs Without Dropout

11.1 Model Setup

Two identical CNN models are created:

- **Model 1:** No dropout
- **Model 2:** With dropout ($P = 0.5$) applied after the first fully connected layer

CNN With Dropout

```python
class CNNWithDropout(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 16, 3)
        self.conv2 = nn.Conv2d(16, 32, 3)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(32 * 6 * 6, 64)
        self.drop = nn.Dropout(0.5)  # 50% dropout
        self.fc2 = nn.Linear(64, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(x.size(0), -1)    # flatten
        x = F.relu(self.fc1(x))
        x = self.drop(x)              # dropout after FC1
        x = self.fc2(x)
        return x
```

Listing 1: Dropout Placement in CNN

11.2 Key Observations

11.2.1 Training Mode (`model.train()`)

- **Without dropout:** Same input always produces the same output
- **With dropout:** Each forward pass produces **different outputs** because different neurons are randomly dropped each time
- Two consecutive forward passes with dropout give non-identical results — 50% of neurons are dropped randomly, and the selection differs each time

11.2.2 Evaluation Mode (`model.eval()`)

- **With dropout:** Outputs become **identical** across multiple forward passes because dropout is turned off
- The model uses all neurons and produces deterministic output

11.3 Loss Curve Comparison

- **Without dropout:** Loss decreases quickly
- **With dropout:** Initial loss is higher, convergence is slower, but the model **generalizes better**
- The dropout model achieves better performance on unseen data despite a seemingly worse training loss curve

12 Summary

- The complete CNN pipeline consists of six stages: data preparation, tensor conversion, convolution layers, flattening, dense layers, and output
- The output size formula is $\frac{H-K+2P}{S} + 1$, applicable to both convolution and pooling
- Convolution creates new features using learnable weights; pooling reduces dimensions without parameters

- Kernel dimensions include input channels: $C_{\text{in}} \times K \times K$ weights per filter
- Weight sharing reduces parameter count; the first FC layer typically has the most parameters
- In PyTorch, `nn.CrossEntropyLoss` internally applies LogSoftmax + NLLLoss, so raw logits should be provided
- Softmax converts logits to class probabilities that sum to 1
- CNN optimization is not fundamentally different from ANN—the difference is in parameter structure
- Double descent is a phenomenon where loss decreases, increases, then decreases again during training
- CNNs can overfit, especially with small datasets or noisy labels (feature overlap between classes)
- Dropout randomly turns off neurons during training with probability P (typically 0.2–0.5)
- Dropout follows a Bernoulli distribution: $\tilde{H} = M \odot H$ where $M \sim \text{Bernoulli}(1 - P)$
- Dropout is applied on dense/FC layers (strongest impact), is ON during training, and OFF during evaluation
- With dropout, each training forward pass produces different outputs; in eval mode, outputs are deterministic
- Dropout leads to higher initial loss but better generalization